Free Monad

eason: 7 · Advanced FP · Category: Functional · Level: Advanced · Build with Waq

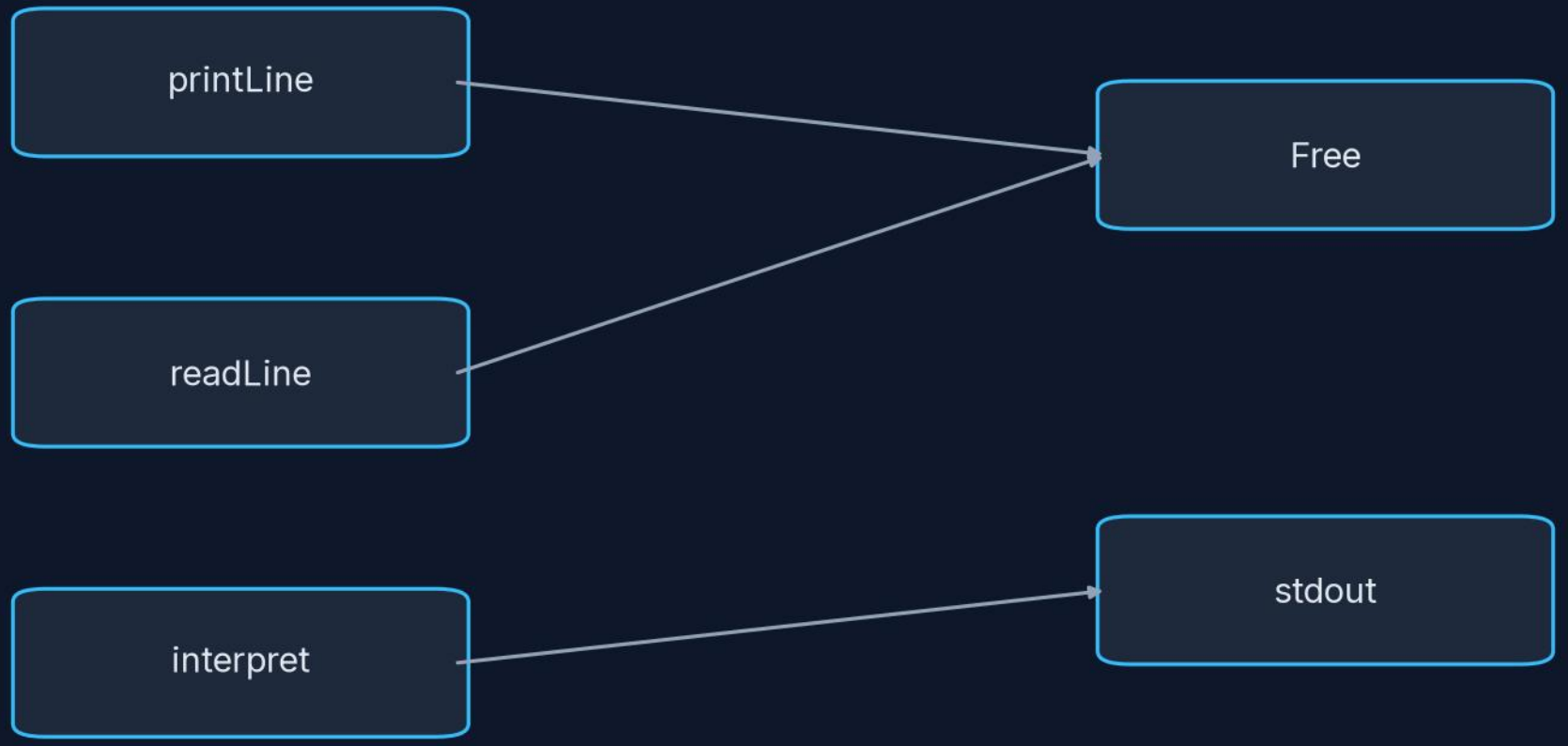
Today's Hook

- Today: Free Monad as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Free Monad structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Free Monad
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/free/Free.java
package com.buildwithwaqar.patterns.functional.free;

import java.util.function.Function;

/** Free monad over a console instruction set (simplified educational encoding). */
public abstract class Free<A> {
    private Free() {}

    public abstract <B> Free<B> flatMap(Function<? super A, Free<B>> f);

    public final <B> Free<B> map(Function<? super A, ? extends B> f) {
        return flatMap(a -> pure(f.apply(a)));
    }

    public static <A> Free<A> pure(A a) { return new Pure<>(a); }

    public static Free<Void> printLine(String s) { return new SuspendPrint(s); }

    public static Free<String> readLine() { return SuspendRead.INSTANCE; }

    private static final class Pure<A> extends Free<A> {
        private final A value;
        Pure(A value) { this.value = value; }
        A value() { return value; }
        public <B> Free<B> flatMap(Function<? super A, Free<B>> f) { return f.apply(value); }
    }
```

Implementation (2)

```
private static final class SuspendPrint extends Free<Void> {
    private final String line;
    SuspendPrint(String line) { this.line = line; }
    public <B> Free<B> flatMap(Function<? super Void, Free<B>> f) {
        return new Bind<>(this, f);
    }
}

private static final class SuspendRead extends Free<String> {
    static final SuspendRead INSTANCE = new SuspendRead();
    public <B> Free<B> flatMap(Function<? super String, Free<B>> f) {
        return new Bind<>(this, f);
    }
}

private static final class Bind<A, B> extends Free<B> {
    private final Free<A> prev;
    private final Function<? super A, Free<B>> f;
    Bind(Free<A> prev, Function<? super A, Free<B>> f) {
        this.prev = prev;
        this.f = f;
    }
```

Implementation (3)

```
        public <C> Free<C> flatMap(Function<? super B, Free<C>> g) {
            return new Bind<>(prev, a -> f.apply(a).flatMap(g));
        }
    }

    @SuppressWarnings({"rawtypes", "unchecked"})
    public static <A> A interpret(Free<A> program, String cannedInput, StringBuilder out) {
        Free cur = program;
        while (true) {
            if (cur instanceof Pure<?> p) {
                return (A) p.value();
            } else if (cur instanceof SuspendPrint sp) {
                out.append(sp.line).append('\n');
                cur = pure(null);
            } else if (cur instanceof SuspendRead) {
                cur = pure(cannedInput);
            } else if (cur instanceof Bind<?, ?> b) {
                Object inner = interpret(b.prev, cannedInput, out);
                Function f = b.f;
                cur = (Free) f.apply(inner);
            } else {
```

Demo.main

```java
package com.buildwithwaqar.patterns.functional.free;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Free Monad walkthrough ===");
    Free<String> program = Free.printLine("name?")
      .flatMap(ignored -> Free.readLine())
      .map(n -> "hi " + n);
    var out = new StringBuilder();
    System.out.println("result=" + Free.interpret(program, "Waqar", out));
    System.out.println("console=" + out.toString().trim());
  }
}
```

Walkthrough Output

```
=== Free Monad walkthrough ===
result=hi Waqar
console=name?
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Free Monad
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Free Monad → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.